Промпт для документации MeetSpace

Ты — технический писатель и backend-архитектор. Проанализируй исходный код MeetSpace и подготовь техническую документацию на русском языке. Оформи её в профессиональный PDF, который можно передать разработчику или системному администратору для изучения архитектуры, установки и сопровождения проекта.

Репозиторий: https://github.com/janickiy/meet-space

Если проект уже открыт в рабочем окружении, используй его исходный код. Зафиксируй анализируемую ветку и commit.

Основной результат — готовый PDF. Дополнительно предоставь редактируемый исходник документации в Markdown и исходники схем.

1. Общие требования

Документация должна описывать фактически реализованный проект.

Источники информации:

- исходный код backend и frontend;
- регистрация маршрутов, обработчики запросов и middleware;
- SQL-миграции, модели и запросы к базе данных;
- Dockerfile и Docker Compose;
- примеры конфигурации и переменных окружения;
- скрипты установки, сборки, запуска и обновления;
- существующая документация и OpenAPI, сверенные с реализацией.

При расхождениях между README и кодом ориентируйся на реализацию. Существенные несоответствия кратко отмечай.

Не переноси сведения из старых проектов go-recorder и go-meeting без проверки. Не придумывай API endpoints, таблицы, поля, команды установки, имена сервисов и возможности проекта.

Если исходники недоступны, запроси архив или доступ к репозиторию. Не заменяй анализ кода предположениями.

Пиши кратко, технически точно и понятным языком. Исключи воду, рекламные формулировки, повторения и общеизвестные определения технологий.

Полноту описания API и базы данных сохраняй за счёт компактных таблиц. Не сокращай документацию путём пропуска таблиц, полей или необходимых шагов установки.

В начале документа укажи репозиторий, ветку и commit. Различай проверку по исходному коду и фактическое выполнение команд. Не заявляй об успешном запуске системы, если он не проводился.

2. Содержание документа

2.1. Титульная страница

Название:

MeetSpace — техническая документация

Подзаголовок:

Архитектура, запись видеопотока, API, база данных и ручная установка

После титульной страницы добавь кликабельное оглавление.

2.2. Назначение и структура проекта

Кратко опиши назначение MeetSpace и основные реализованные подсистемы.

Покажи структуру значимых директорий в таблице:

Путь	Назначение	Ключевые файлы

Отрази расположение:

- точек входа сервисов;
- API и бизнес-логики;
- компонентов WebRTC и записи;
- frontend;
- миграций;
- инфраструктурной конфигурации;
- скриптов установки и запуска;
- документации.

Используй реальные пути. Не перечисляй зависимости, кеши, сгенерированные сборки и каждый исходный файл.

2.3. Архитектура и роли компонентов

Создай общую архитектурную схему и таблицу:

Компонент	Реальное имя сервиса или модуля	Ответственность	Взаимодействие

Обязательно исследуй и отрази:

- браузерный клиент;
- API;
- WebRTC и сигналинг;
- Go recorder-worker;
- FFmpeg;
- RabbitMQ;
- PostgreSQL;
- локальный storage volume.

Название «Go recorder-worker» используй как обозначение роли компонента записи. Укажи соответствующее ему фактическое имя сервиса, процесса или пакета.

Если в реализации присутствуют media-worker/SFU, Redis, MinIO/S3, TURN/STUN, reverse proxy или дополнительные workers, включи их в схему и кратко объясни назначение.

На схеме раздели и подпиши:

- передачу аудио и видео;
- HTTP-запросы и WebSocket-сигналинг;
- команды и события RabbitMQ;
- работу с PostgreSQL;
- чтение и запись файлов.

Укажи протоколы и направления передачи данных. Добавь компактную легенду.

2.4. Запись видеопотока из браузера

Это один из основных разделов документа.

Проследи реальную цепочку записи от браузера до доступного пользователю файла.

Объясни по шагам:

1. Получение браузером камеры, микрофона или демонстрации экрана.
2. Установление WebRTC-соединения и обмен сигналингом.
3. Поступление медиаданных в принимающий компонент.
4. Запуск записи через API и передачу команды worker.
5. Получение команды Go recorder-worker и подготовку записи.
6. Передачу медиаданных в FFmpeg.
7. Создание промежуточных сегментов или файлов.
8. Остановку и финализацию записи.
9. Сохранение метаданных и статусов в PostgreSQL.
10. Получение пользователем готовой записи.

Уточни каждый шаг по исходникам. Если используется media-worker/SFU или другой промежуточный компонент, обязательно покажи его.

Отдельно опиши:

- что передаётся через RabbitMQ;
- каким путём передаются сами медиаданные;
- как recorder-worker взаимодействует с FFmpeg;
- какие входные и выходные форматы используются;
- где находятся временные и итоговые файлы;
- какие сервисы монтируют storage volume;
- как пути внутри контейнеров связаны с хранилищем на сервере;
- когда выполняется очистка промежуточных данных;

- как обрабатываются остановка, ошибка и повторная команда — в пределах реализованного поведения.

Если финальные файлы сохраняются в MinIO/S3, объясни его роль отдельно от локального volume. Не представляй временное хранилище как единственное постоянное хранилище.

Создай:

- схему движения медиаданных и команд;
- sequence diagram запуска, выполнения и завершения записи;
- компактную таблицу фактических статусов записи и условий перехода между ними.

Не добавляй неподтверждённые возможности: композитную запись, смешивание аудио, автоматическое восстановление или повторную обработку, если их нет в коде.

2.5. API endpoints

Составь полный каталог существующих API endpoints и сгруппируй их по назначению.

Для каждого endpoint укажи:

- HTTP-метод и полный путь с учётом префикса;
- назначение;
- требования к авторизации и роли;
- параметры path и query;
- обязательные и существенные поля тела запроса;
- основные данные ответа и код успешного результата;
- значимые ошибки.

Используй компактные таблицы. Общие правила авторизации, форматы ошибок и повторяющиеся параметры опиши один раз.

Различай публичные, пользовательские, административные и внутренние endpoints, если такие группы существуют.

Для основных сценариев записи приведи реальные примеры cURL-запросов и JSON-ответов:

- запуск записи;
- остановка записи;
- получение состояния;
- просмотр списка;
- получение или скачивание файла.

Используй фактические маршруты. Если отдельного endpoint для действия нет, опиши существующий механизм.

Если используется WebSocket, отдельно опиши адрес подключения, авторизацию и основные реализованные типы сообщений.

Примеры должны соответствовать обработчикам и структурам данных. Секреты, токены и адреса окружения заменяй понятными placeholders.

2.6. База данных

Опиши итоговую схему PostgreSQL после применения всех актуальных миграций.

Включи все действующие таблицы проекта: прикладные, связующие и служебные. Системные каталоги PostgreSQL описывать не нужно.

В начале раздела приведи перечень:

Таблица	**Назначение**

Затем для каждой таблицы опиши каждое поле:

Поле	**Тип**	**NULL**	**Значение по умолчанию**	**Назначение**

Для каждой таблицы также укажи:

- первичный ключ;
- внешние ключи и связанные таблицы;
- уникальные ограничения;
- индексы и их назначение;
- существенные CHECK-ограничения;
- правила удаления связанных записей, если они заданы.

Расшифруй enum, статусы и значимые значения полей. Для JSON/JSONB кратко опиши структуру, если она определяется кодом.

Учитывай переименования, удаления и изменения полей в последующих миграциях. Не описывай старую схему как действующую и не ограничивайся ORM-моделями.

Создай читаемую ER-диаграмму. Если таблиц много, раздели её по подсистемам. Отличай реальные внешние ключи от логических связей, которые поддерживаются только приложением.

Проверь полноту: каждая действующая таблица и каждое поле должны присутствовать в документации.

2.7. Ручная установка

Подготовь последовательную инструкцию для чистого сервера на поддерживаемой проектом ОС.

Ручная установка означает выполнение понятных команд администратором. Если проект рассчитан на Docker Compose, используй этот способ и назови его ручным развёртыванием через Docker Compose.

Инструкция должна включать:

1. Требования к серверу и необходимое ПО.
2. Получение исходников или предусмотренного проектом релиза.
3. Подготовку конфигурации.
4. Настройку обязательных переменных окружения.
5. Создание каталогов и настройку storage volume.
6. Подготовку PostgreSQL, RabbitMQ и остальных необходимых сервисов.

7. Применение миграций.
8. Сборку и запуск компонентов.
9. Настройку домена, HTTPS, reverse proxy и TURN/STUN, если требуется.
10. Проверку работоспособности.

Приводи команды в порядке выполнения. Указывай рабочую директорию, реальные имена файлов, Compose-профилей и сервисов.

Для основных переменных окружения сделай таблицу:

Переменная	**Назначение**	**Обязательность**	**Пример**

Используй примеры конфигурации из проекта. Не раскрывай реальные пароли, токены и другие секреты.

Отдельно перечисли необходимые сетевые порты с протоколом и назначением. Раздели внешние подключения и внутреннее взаимодействие сервисов.

Проверка после установки должна охватывать:

- доступность приложения и API;
- подключение участников;
- передачу аудио и видео;
- запуск и завершение записи;
- появление метаданных в БД;
- сохранение и получение готового файла.

Не придумывай отсутствующий install.sh, неподдерживаемые параметры запуска и команды миграций.

Если предусмотрено несколько способов развёртывания, подробно опиши один основной воспроизводимый вариант.

2.8. Эксплуатация и диагностика

Добавь короткий практический раздел:

- просмотр состояния и логов сервисов;
- расположение конфигурации и файлов записей;
- безопасная остановка и перезапуск с учётом активных записей;
- контроль свободного места и очистка временных файлов;
- диагностика основных проблем.

Типовые проблемы оформи таблицей:

Симптом	**Что проверить**	**Команда или место проверки**

Включи только относящиеся к реализации ситуации:

- отсутствие аудио или видео;
- запись не запускается;
- ошибка FFmpeg;

- недоступность RabbitMQ или PostgreSQL;
- недостаточные права на storage volume;
- нехватка места;
- отсутствие итогового файла.

Команды очистки сопровождай точным указанием затрагиваемых данных. Не предлагай удаление рабочих volumes как обычный способ устранения ошибок.

Перед заключительным разделом кратко перечисли основные файлы исходного кода, на которых основана документация.

3. Оформление PDF

Сделай аккуратный современный технический документ.

Требования:

- формат A4;
- единая типографика и спокойная цветовая палитра;
- шрифты с поддержкой кириллицы, встроенные в PDF;
- титульная страница;
- кликабельное оглавление и PDF-закладки;
- последовательная нумерация разделов и страниц;
- читаемые таблицы с повторением заголовков при переносе;
- моноширинный шрифт для кода, команд, путей и идентификаторов;
- схемы в векторном формате или достаточном разрешении;
- активные ссылки;
- текст, доступный для поиска и копирования.

Размер текста и схем должен позволять читать документ без постоянного увеличения.

Широкие таблицы перестраивай или размещай на альбомных страницах. Не уменьшай шрифт до нечитаемого размера.

Избегай обрезанных строк, наложений, пустых декоративных страниц, оторванных заголовков и больших неиспользуемых участков.

Схемы Mermaid или другого инструмента должны попасть в PDF как отрисованные диаграммы.

4. Заключительный раздел — техническая поддержка

В конце документа размести следующий текст, сохранив формулировки, дату и контакты. Оформи заголовки, абзацы и кликабельные контакты.

Техническая поддержка

Если у вас возникли вопросы по установке, настройке или использованию MeetSpace, вы можете обратиться за технической поддержкой.

Документация на русском · Проверено 20 октября 2026 г.

Индивидуальная доработка MeetSpace

Я также предлагаю услуги по индивидуальной доработке и адаптации MeetSpace под конкретные задачи и требования заказчика. Возможна разработка дополнительного функционала, интеграция со сторонними сервисами, изменение интерфейса, оптимизация производительности и настройка серверной инфраструктуры.

Каждый запрос рассматривается индивидуально с учётом особенностей проекта и пожеланий заказчика.

Контакты

По вопросам технической поддержки, установки, настройки и индивидуальной разработки обращайтесь:

Email: janickiy@mail.ru

Telegram: @janickiy

5. Проверка и выдача результата

Перед завершением:

1. Сверь каталог API с регистрацией маршрутов.
2. Сверь таблицы и поля БД с итоговой схемой миграций.
3. Проверь соответствие схем реальным путям медиаданных, команд и файлов.
4. Проверь команды установки по конфигурации и скриптам проекта.
5. Собери PDF, отрендери его страницы и визуально проверь оформление.
6. Исправь обрезку, переносы, нечитаемые схемы и ошибки кириллицы.
7. Проверь наличие заключительного блока и правильность контактов.

Предоставь:

- `MeetSpace_Technical_Documentation_RU.pdf`;
- `MeetSpace_Technical_Documentation_RU.md`;
- исходники диаграмм.

В ответе дай ссылки на готовые файлы и кратко перечисли содержимое документа. Существенные неподтверждённые моменты укажи отдельно.

Задача считается выполненной после создания и проверки итогового PDF.